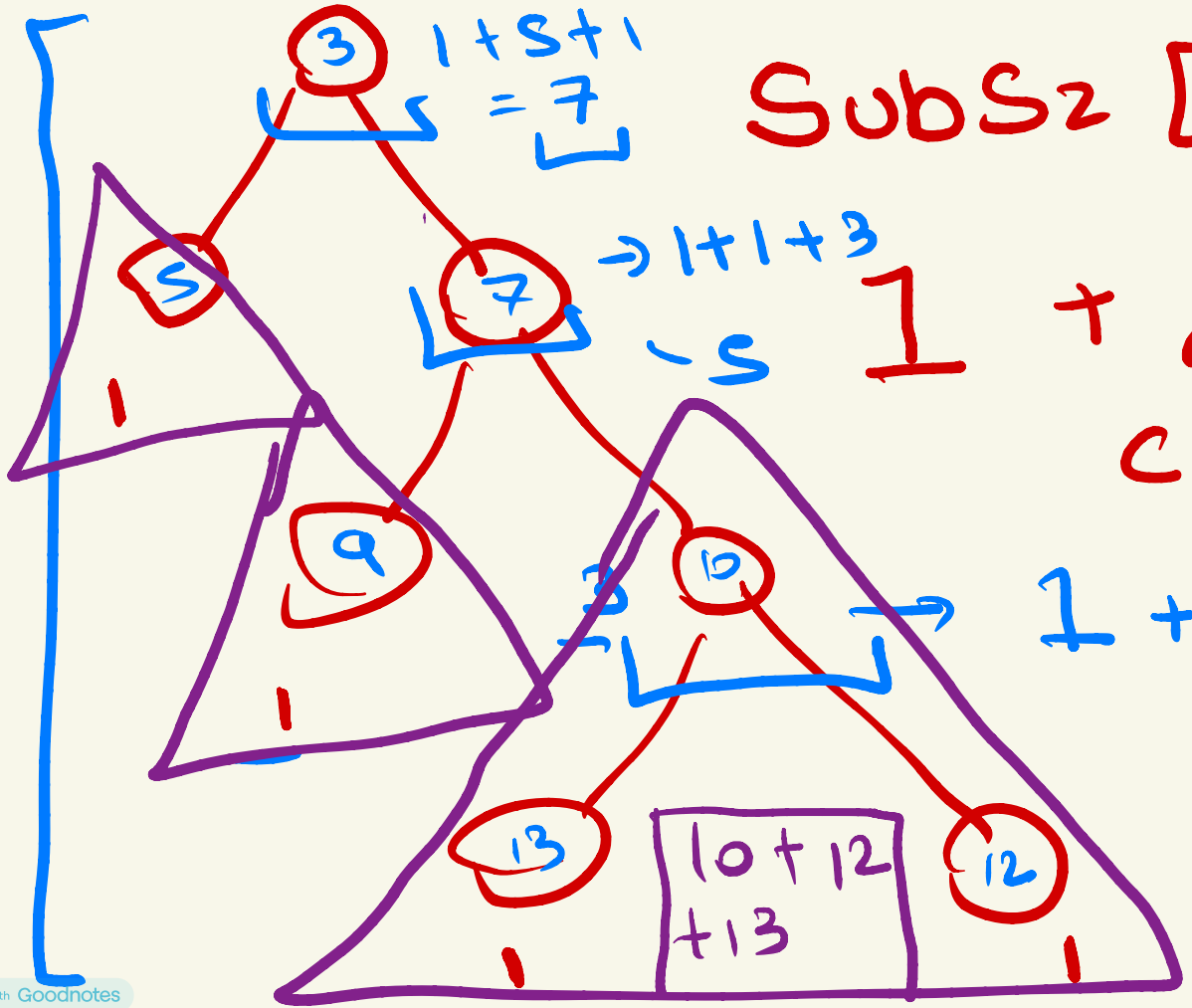
DP on Trees

- Gaurish

Standard Techniques:

1. Size of Subtree
2. Sum of Values in Subtree
3. Diameter of Tree (Previously covered)



$$SubS_2[node] = 1 + \sum_{c \in children_{node}} SubS_2[c]$$

?

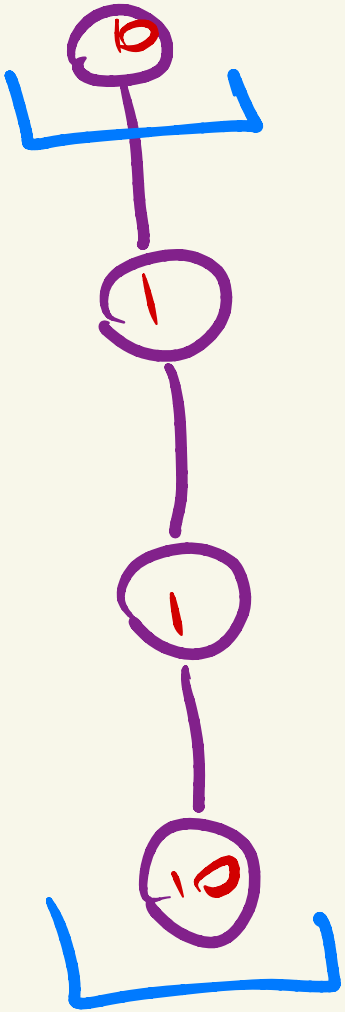
```
dfs (node, par) {
    s2[node] = arr[node]
    for i ∈ adj[node]
        if (i ≠ par) {
            → dfs(i, node)
              s2[node] += s2[i]
    }
}
```

s2 → sum of values in subtree

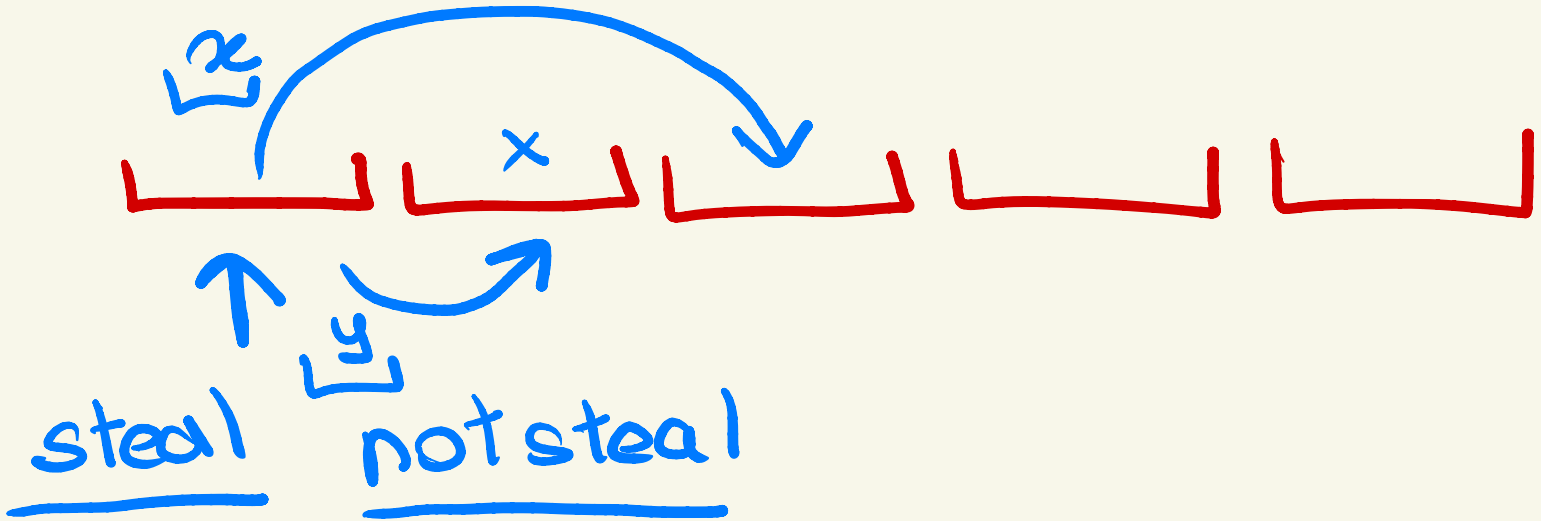
$$DP[node] = arr[node] + \sum_{C \in children_{node}} DP[C]$$

Problem 1: House Robber

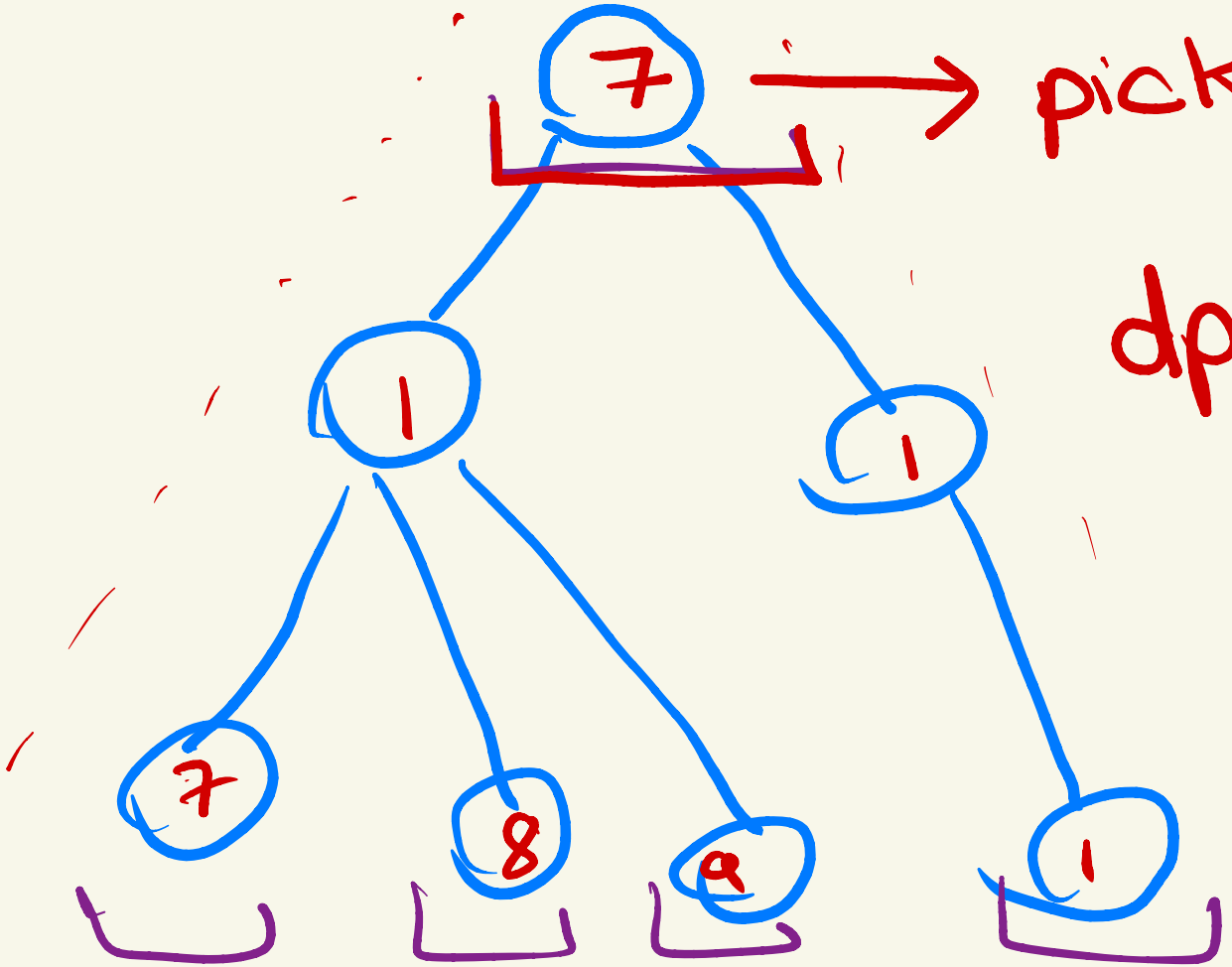
Given a tree T of N nodes, where each node has Ci coins. You need to choose a subset of nodes such that every pair of nodes in this subset is not connected by an edge. Find the maximum number of coins you can get.



10 + 10 = 20



Solved it for array



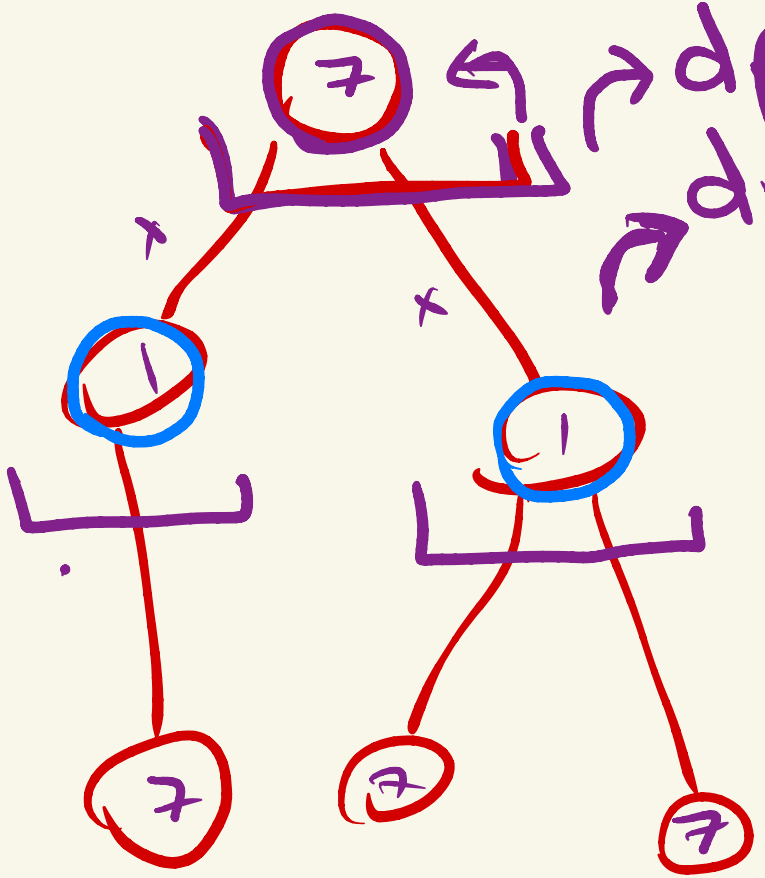
pick don't pick

dp[i][1]

↳ max ans when I pick

dp[i][0]

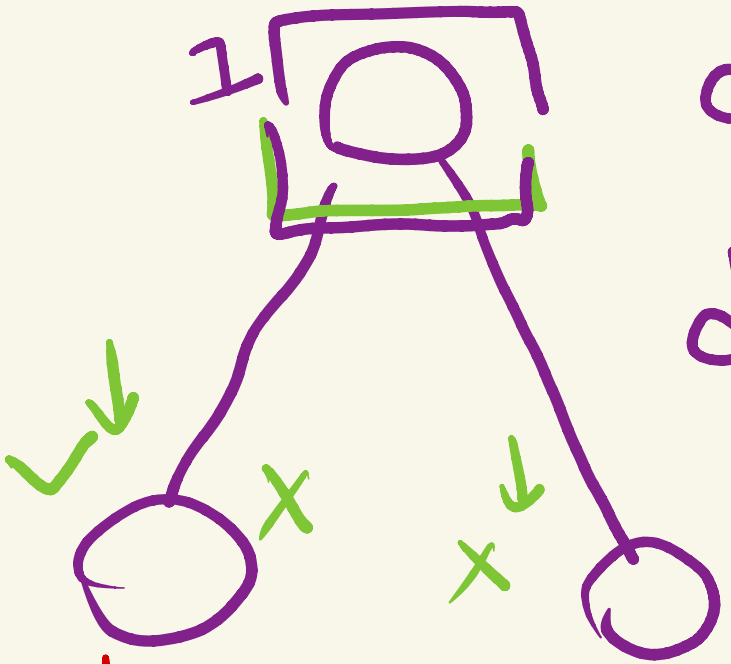
↳ don't pick



$$dp[i][1] = arr[i]$$

$$dp[i][0] = 0$$

$$dp[i][1] = arr[i] + \sum_{j \in Children_i} dp[j][0]$$



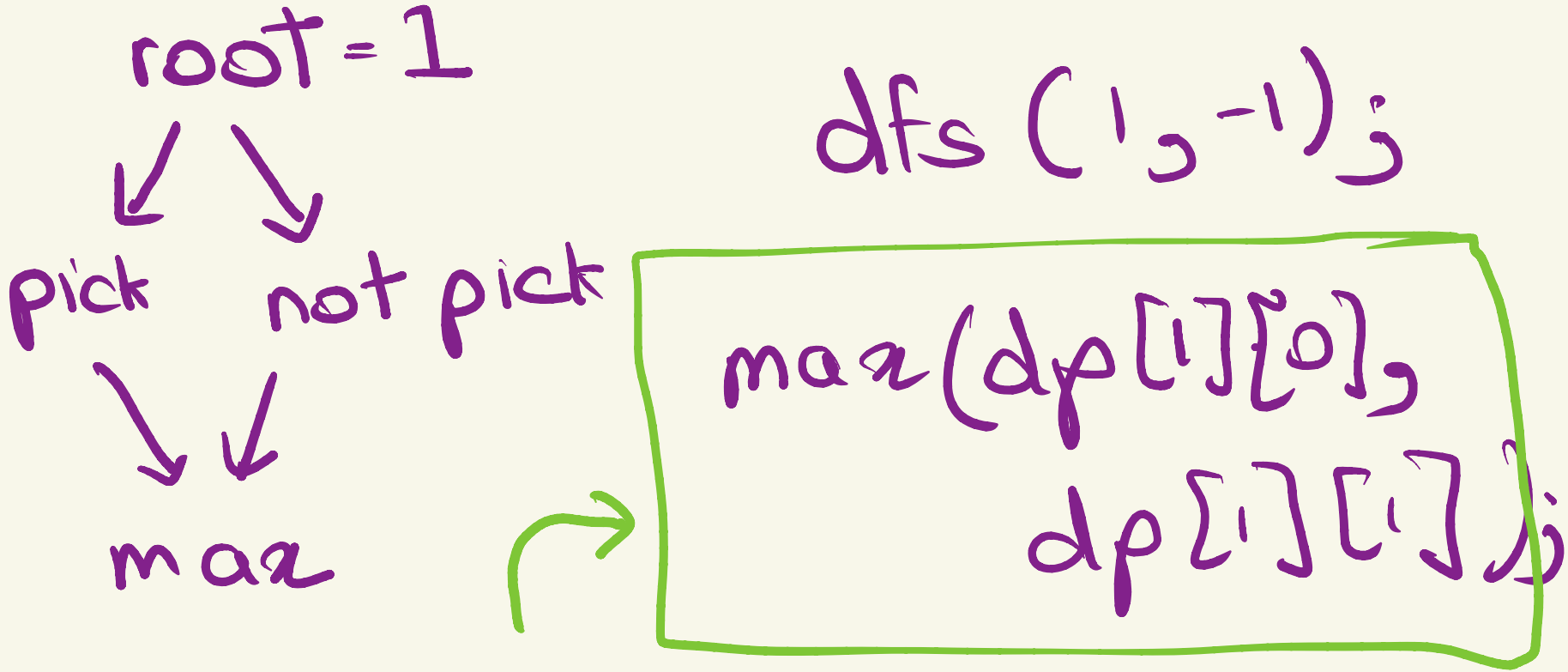
$dp[i][0] = 0$

$dp[i][0] =$

$$\sum_{j \in \text{children}} \max(dp[j][1], dp[j][0])$$

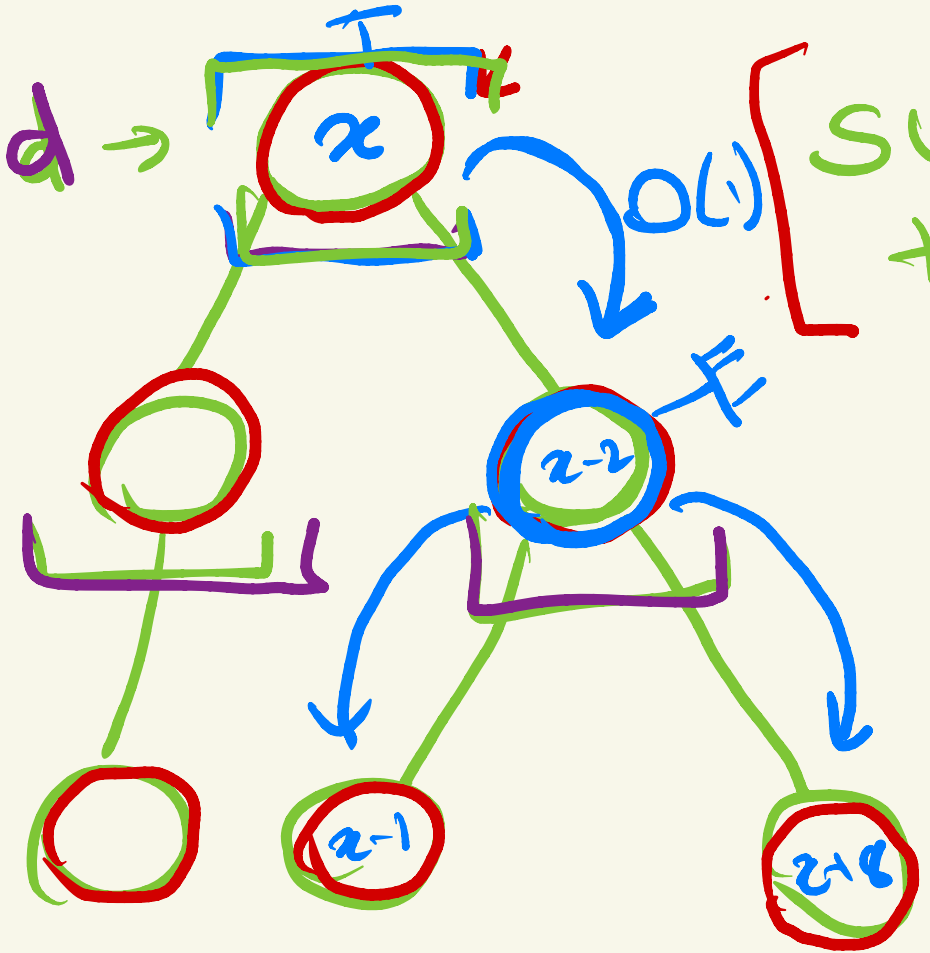
↳ $dp[j][0]$

$dp[j][1]$ → max



Rerooting DP

1. When you have to calculate the answer for each root.
2. Steps to do re-rooting dp:
3. Assume any node as the root and calculate the answer
4. Now While traversing from old root to new root, calculate how answer changes
5. Solve recursively



sum of distances to other nodes.

$O(n^2)$ $n \leq 10^5$

Brute TLE ✗

Tree Distances II

https://cses.fi/problemset/task/1133/

CSES Problem Set

Tree Distances II

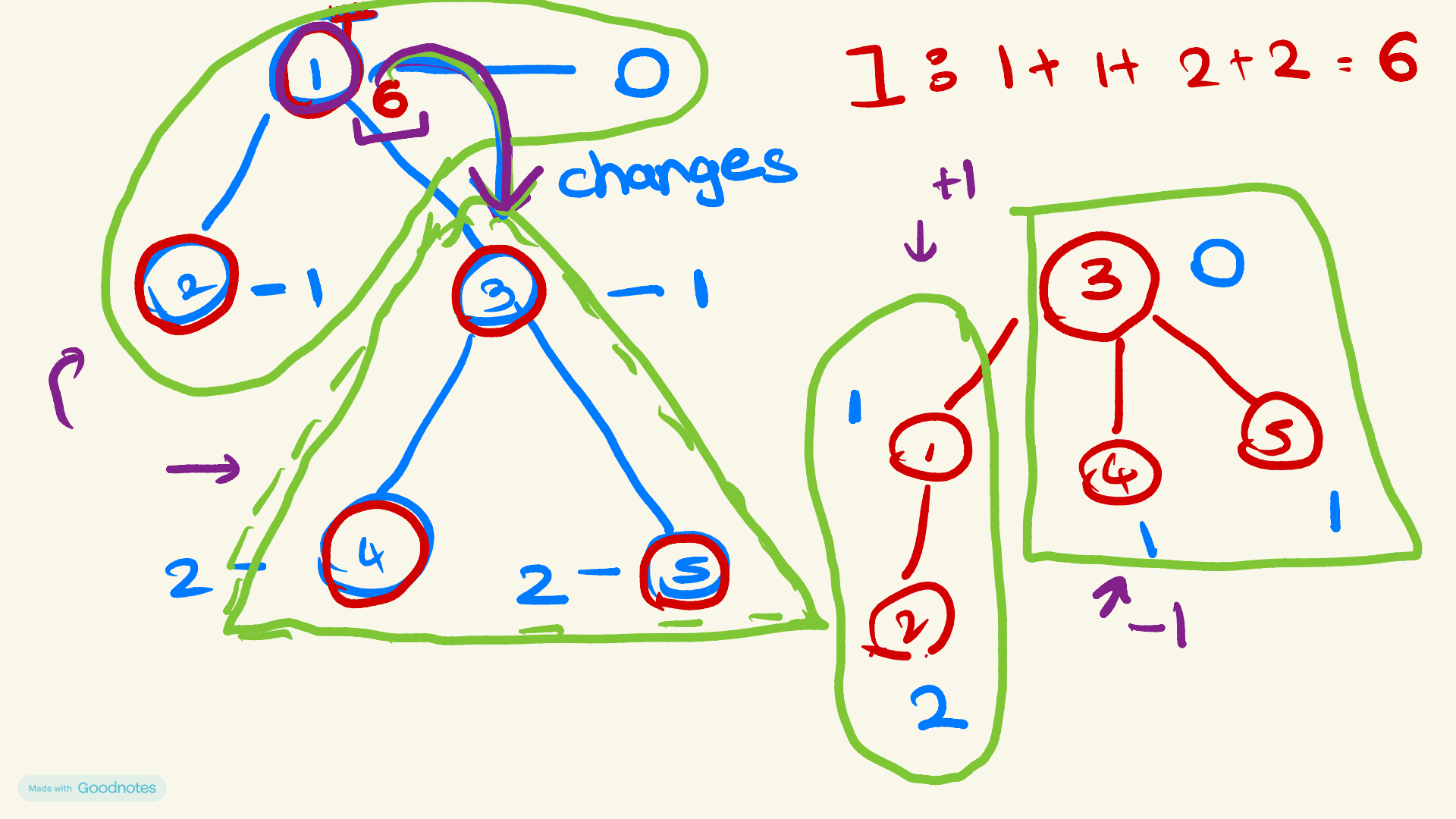
TASK | SUBMIT | RESULTS | STATISTICS | TESTS | QUEUE

Time limit: 1.00 s **Memory limit:** 512 MB

You are given a tree consisting of n nodes.

Your task is to determine for each node the sum of the distances from the node to all other nodes.

Input


1 : 1 + 1 + 2 + 2 = 6
1
6
0
changes
2
-1
3
-1
4
2
5
2
+1
1
1
2
2
3
0
4
5
1
-1

$$dp_{newroot} = dp_{root} - S^2_{newroot} + (n - S^2_{newroot})$$

$$dp_{newroot} = dp_{root} + n - 2 \times S^2_{newroot}$$

Tree XOR

https://codeforces.com/contest/1882/problem/D

D. Tree XOR

time limit per test: 3 seconds

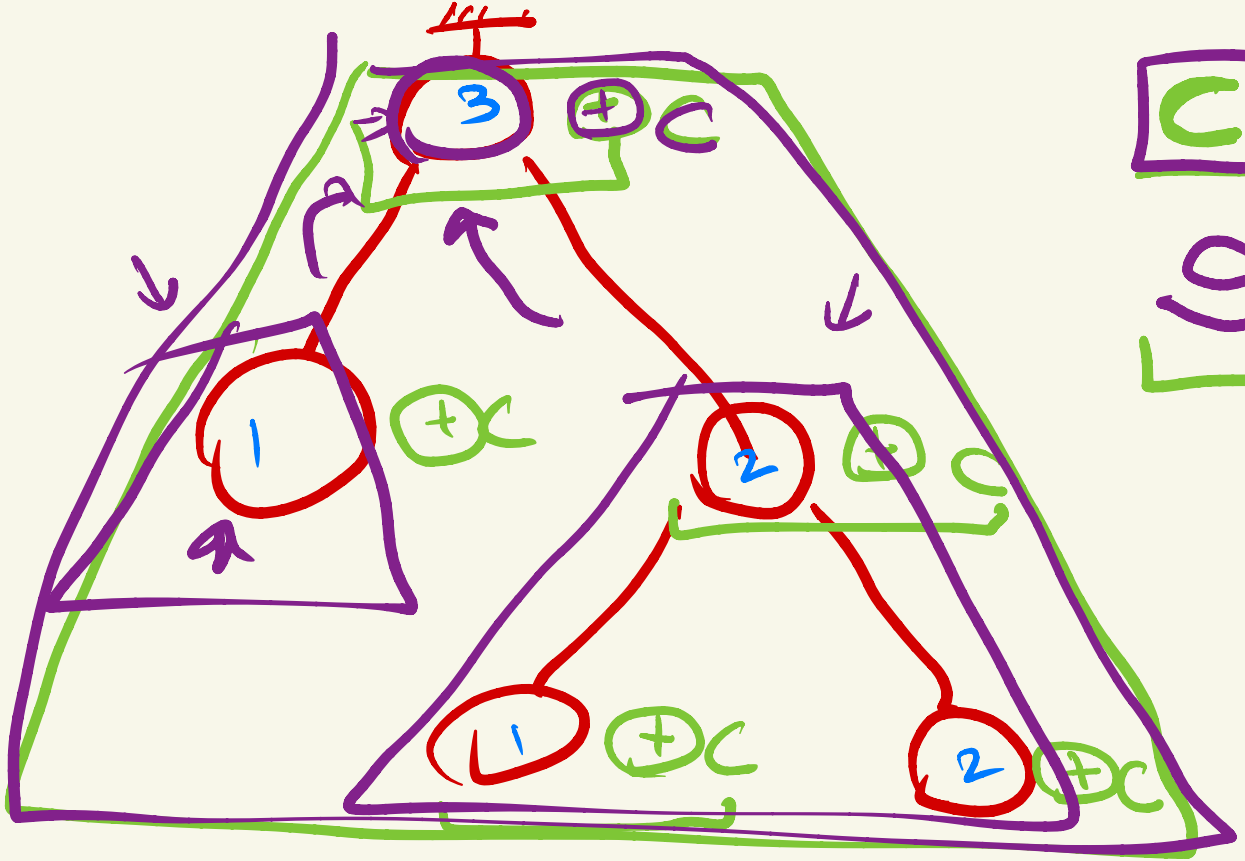
memory limit per test: 512 megabytes

You are given a tree with n vertices labeled from 1 to n. An integer a_i is written on vertex i for $i = 1, 2, \ldots, n$. You want to make all a_i equal by performing some (possibly, zero) spells.

Suppose you root the tree at some vertex. On each spell, you can select any vertex v and any non-negative integer c. Then for all vertices i in the subtree[†] of v, replace a_i with $a_i \oplus c$. The cost of this spell is $s \cdot c$, where s is the number of vertices in the subtree. Here $\oplus$ denotes the bitwise XOR operation.

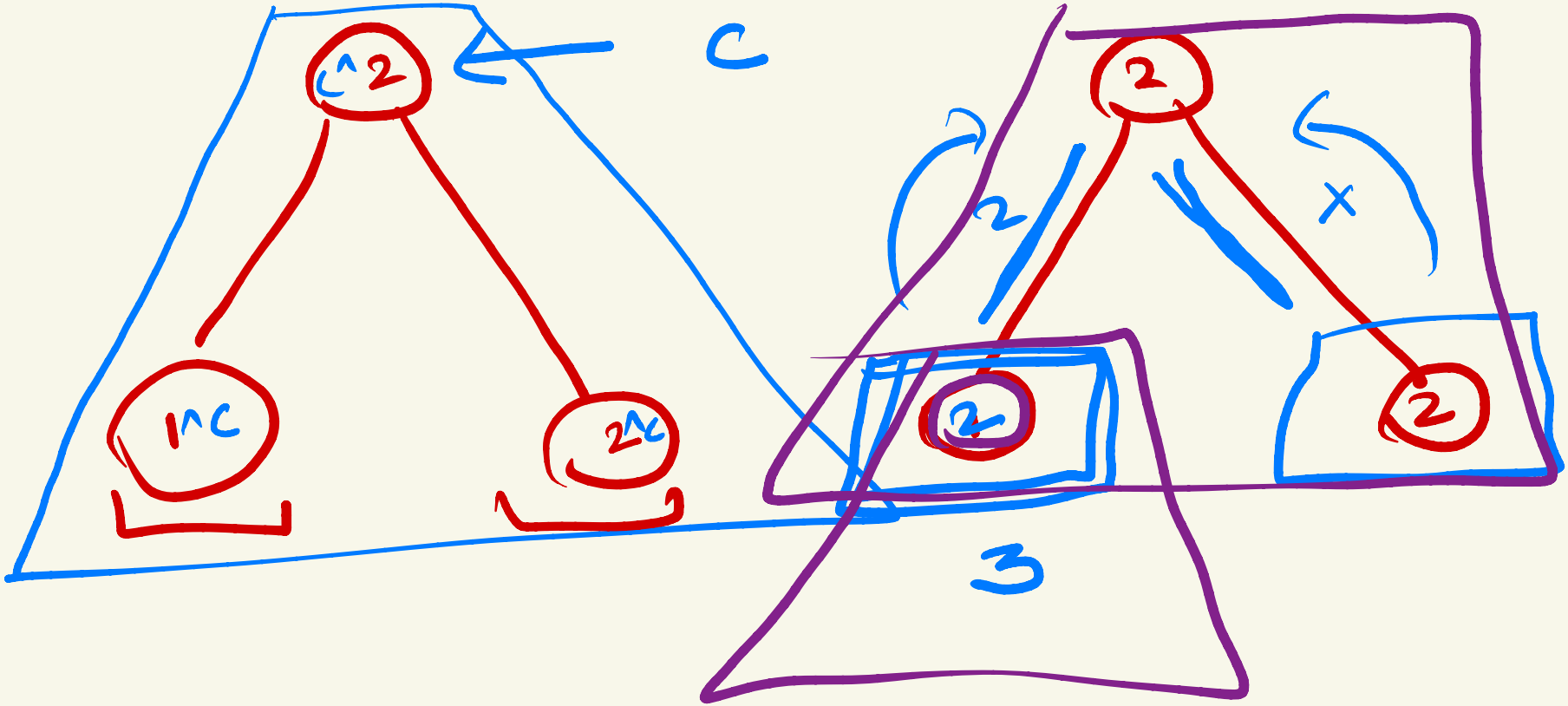
Let m_r be the minimum possible total cost required to make all a_i equal, if vertex r is chosen as the root of the tree. Find $m_1, m_2, \ldots, m_n$.

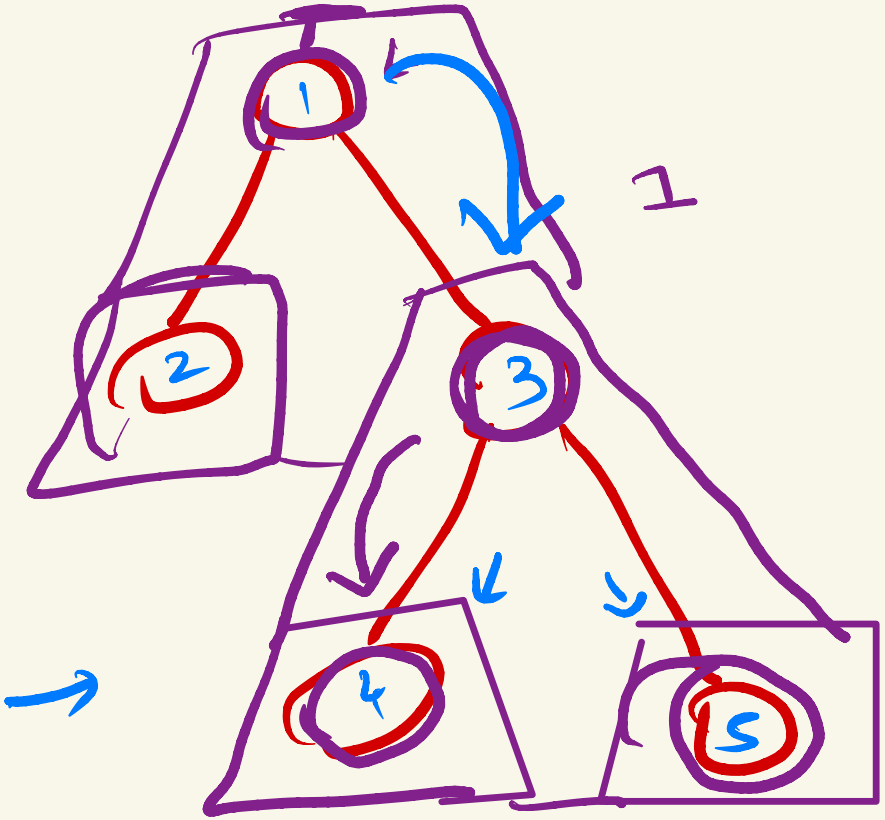
[†] Suppose vertex r is chosen as the root of the tree. Then vertex i belongs to the subtree of v if the simple path from i to r contains v.



C

$S_2 \cdot C$





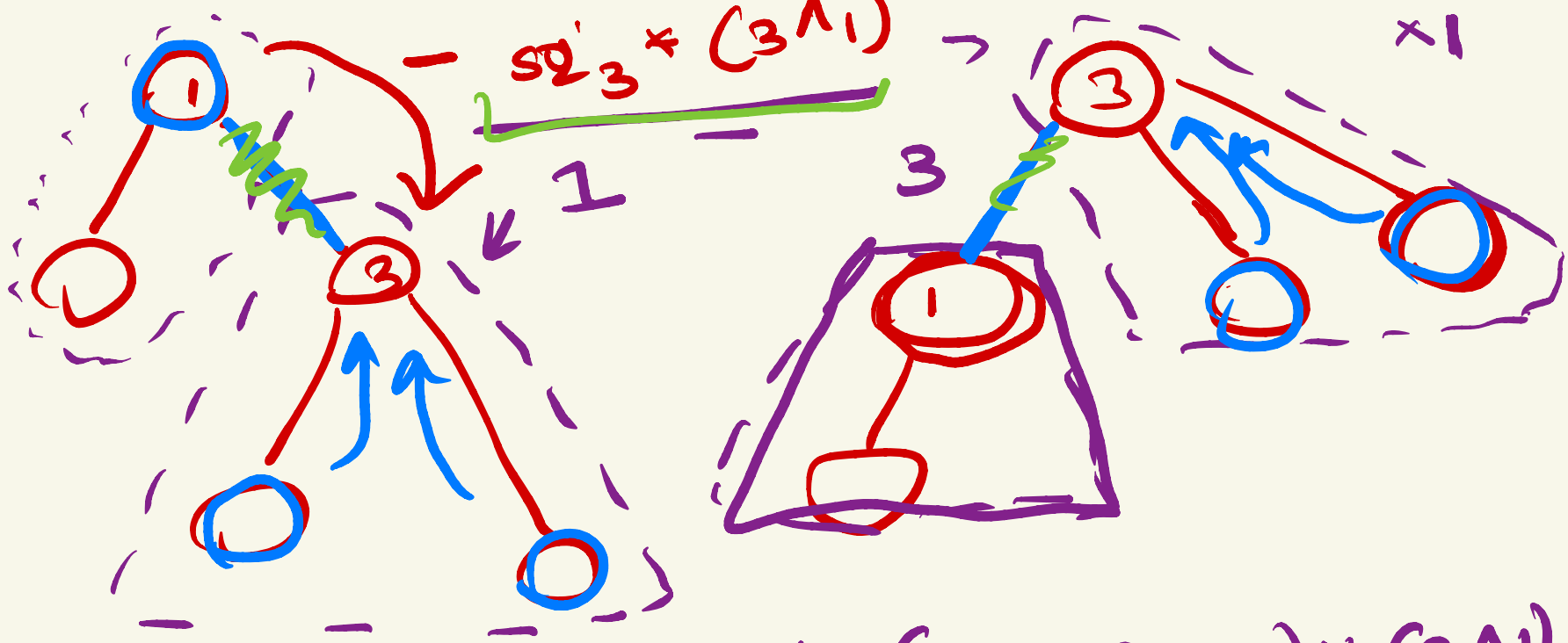
1

$4 \oplus C = 3$

$C = 3 \oplus 4$

$S_2[4] * C$

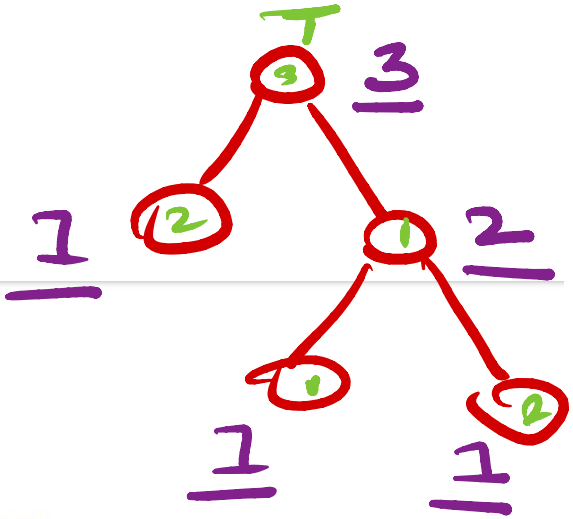
root => $\sum S_2[node] * (all_{node} \oplus or\ (par))$



$$ans_3 = ans_1 = (3^1)\,sz_3 + (n - sz_3)(3^1) + (n - sz_3) * (3^1)$$

$$ans_3 = ans_1 + \underbrace{(3\wedge 1)}_{\times} \underbrace{(n - 2\ s\ z_3)}$$

Small To Large Merging



CSES Problem Set

Distinct Colors

TASK | SUBMIT | RESULTS | STATISTICS | TESTS | QUEUE

Time limit: 1.00 s **Memory limit:** 512 MB

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a color.

Your task is to determine for each node the number of distinct colors in the subtree of the node.

```
set<int> s[n+1];

dfs(node, par) {                                   O(n)
    s[node].insert(arr[node]);
    for i in adj[node]:
        if i ≠ pare
            dfs(i, node)
            for (auto dj : s[i])                   O(n)
                s[node].insert(j);                 O(log n)
    }
ans[node] = s[node].size();
```

TLE

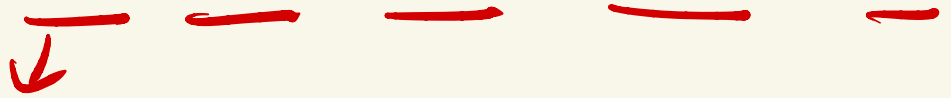
$O(n^2 \log n)$

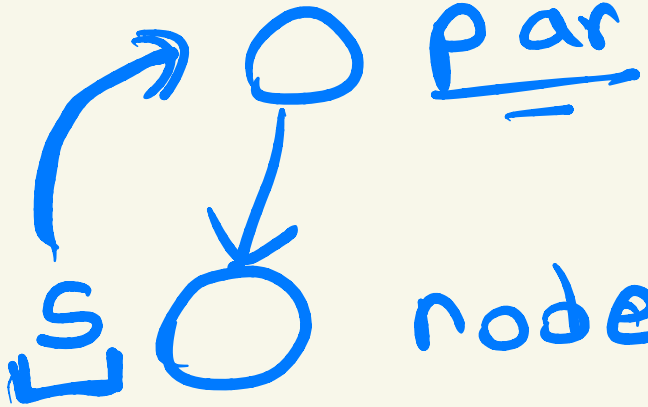
swap (s[i] , s[j])

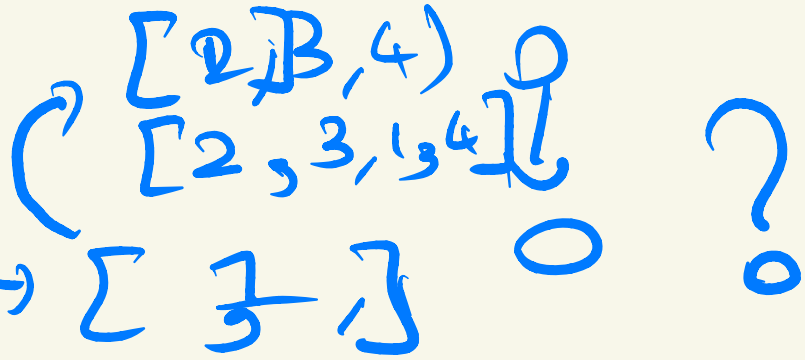
O(1)

Set<int> s[n]

pointer to asct







$|S_{node}| < |S_{par}|$

$|S_{par}| < |S_{node}|$ Swap(Snode)

$O(1)$

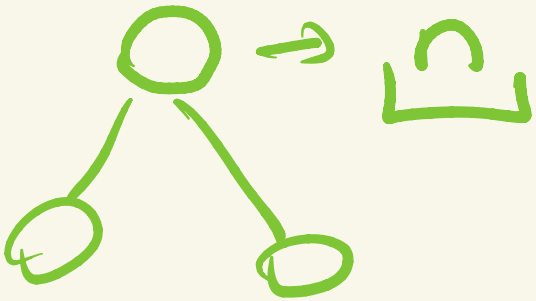
$O(n^2 \log n)$ ⟸

[ ] ≥ 4

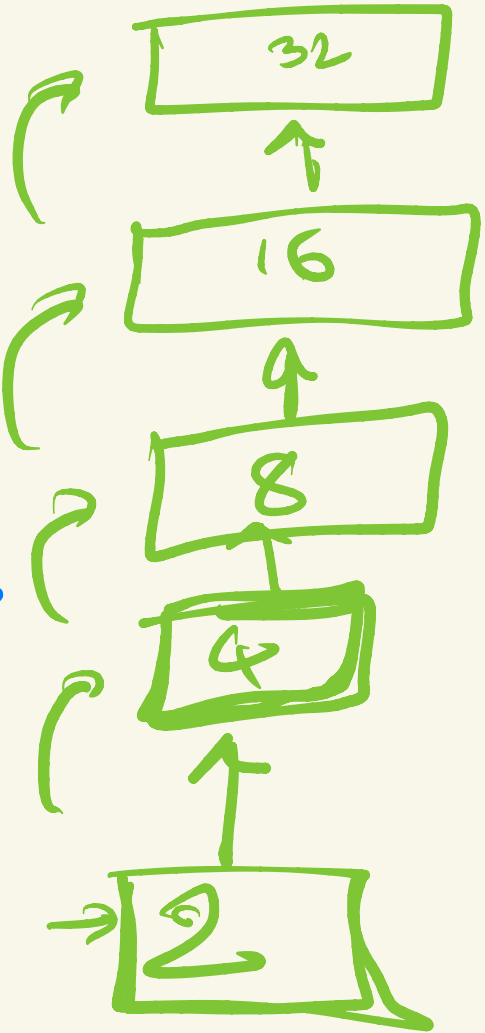
[- -] ≥ 2 ≥ 2

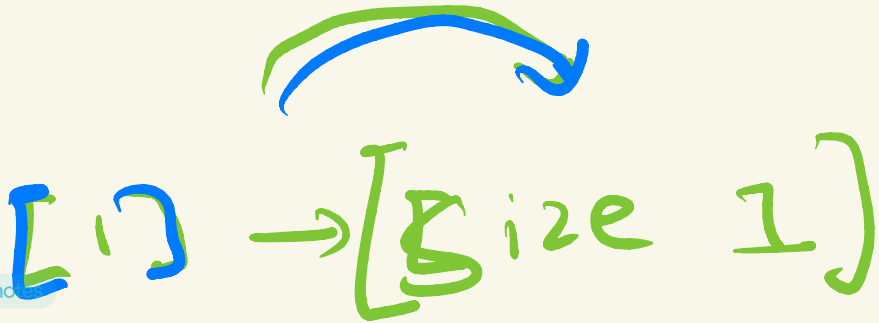
O [1] 1

$\log_2 n$



$n \log_2 n$ inserts

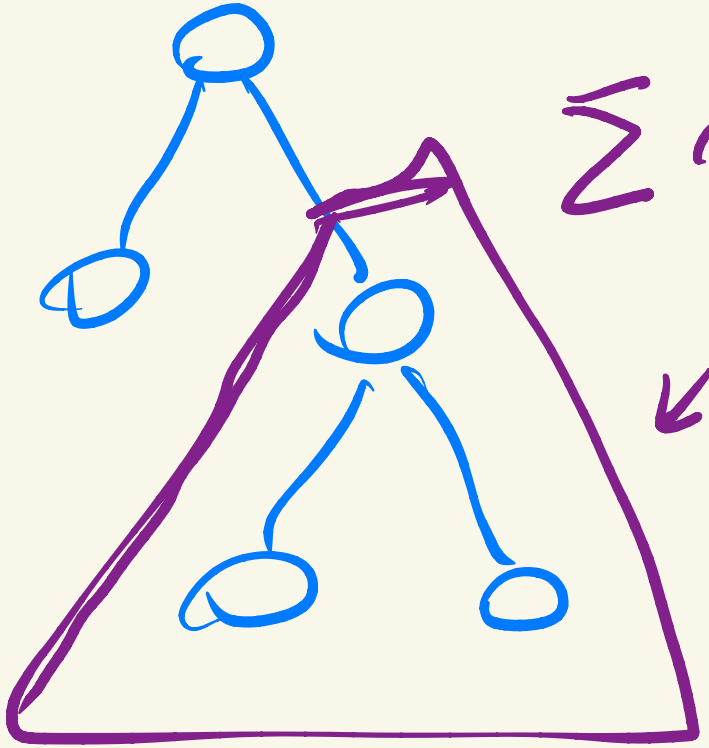




$$n \cdot \log_2 n \cdot \log_2 n$$

$$\downarrow$$

$$O(n \cdot \log_2^2(n))$$



$\sum x \cdot d(f_x)^2$

↙ mapcint, intsm